

3. **The cloud:** The ability to stream and store data over the Internet, visualise it in a Web browser and control the device remotely from any part of the world is all because of the cloud, which virtually makes the data available anytime, anywhere.

There are innumerable ways to get into the IoT space, right away. In this article, I'll talk about communication protocols for the IoT space, which can be used for communication between machines or between a machine and server. Due to constraints in processing capabilities and the low power requirements of IoT devices (which are generally meant to be deployed in environments with constant battery power) with limited bandwidth capabilities, a need was felt for dedicated standards and protocols especially designed for IoT. Since those who manufacture IoT devices and those who create the IoT platforms are different, this required industry standards and protocols that were not high on power consumption, bandwidth usage, or processing power and could be adopted easily by all IoT players—hardware manufacturers, software developers or cloud solutions/service providers.

When developing and deploying an IoT project, it's important to answer questions like:

- How do my devices talk to each other or to me?
- Do I want the stability of a wired network or the freedom of a wireless one?
- What are my constraints? Data rates, battery power or poor networks?
- What communication options do I have?

Enter the world of IoT communications

This section covers a list of IoT communication protocols.

1. MQTT (Message Queue Telemetry Transport)

MQTT is my preferred IoT protocol, which I use for almost all my IoT automation projects. It was created about 15 years back for monitoring remote sensor nodes, and is designed to conserve both power and memory. It is based on the 'Publish Subscribe' communication model, where a broker is responsible for relaying messages to MQTT clients. This allows multiple clients to post messages and receive updates on different topics from a central server known as the MQTT broker. This is similar to subscribing to a YouTube channel, where you get notified whenever a new video is posted.

Using MQTT, a connected device can subscribe to any number of topics hosted by an MQTT broker. Whenever a different device publishes data on any of those topics, the server sends out a message to all connected subscribers of those topics, alerting them to the new available data. It

is overall a lightweight protocol that runs on embedded devices and mobile platforms, while connecting to highly scalable enterprise and Web servers over wired or wireless networks. It is useful for connections with remote embedded systems, where a small code footprint is required and/or network bandwidth is at a premium or connectivity is unpredictable. It is also ideal for mobile applications that require a small size, low power usage, minimised data packets, and efficient distribution of information to one or many receivers. It is an ISO standard (ISO/IEC PRF 20922) protocol. The good performance and reliability of MQTT is demonstrated by Facebook Messenger, Amazon IoT (AWS-IoT), IBM Node-Red, etc—organisations that are using it to serve millions of people daily.

An MQTT-SN or MQTT sensor network allows you to use MQTT over a wireless sensor network, which is not generally a TCP/IP based model. The MQTT broker can be run locally or deployed on the cloud. It is further enhanced with features like user name/password authentication, encryption using Transport Layer Security (TLS) and Quality of Service (QoS).

MQTT implementation: MQTT can be implemented with a broker and MQTT clients. The good news is that both can be found open sourced in the Mosquitto package, which is an open source MQTT broker available as a package for Linux, OSX or Windows machines. It runs an MQTT broker daemon, which listens for MQTT translations on Port 1883 of TCP (by default). To install it on Debian based machines (like Ubuntu 16.04, Raspbian Jessie, etc), simply run the following command from the terminal:

```
# sudo apt-get install mosquitto mosquitto-clients
```

This will install and run MQTT broker on your Debian

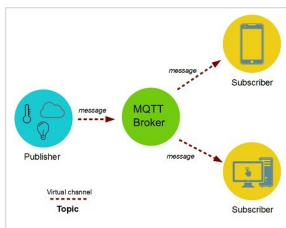


Figure 2: The MQTT model

based Linux machine and provide clients the utilities *mosquitto_pub* and *mosquitto_sub*, which can be used to test and use it.

On the device/client side, Eclipse IoT provides a great open sourced implementation of MQTT and MQTT-SN version 3.1.1 in the form of a library known as Eclipse PAHO, which is available for almost all modern programming languages like C, C++, Java, Python, Arduino, etc, or can be used over WebSockets. For more details or the API reference, visit <http://www.eclipse.org/paho/>.

The table in Figure 3 compares HTTP and MQTT, clearly showing why the latter is a winner in the IoT space.

2. CoAP (Constrained Application Protocol)

Constrained Application Protocol (CoAP) is an Internet application protocol for constrained devices (defined in RFC 7228). It enables constrained devices to communicate with